Commitment Ledger Specification

Project Name

`commitment-ledger`

One-Sentence Description

Commitment Ledger tracks commitments made against available work, records evidence of progress, and preserves outcomes over time.

Core Principle

We are not tracking tasks.

We are tracking commitments against work.

---

1. Vision

Many team repositories already contain `TODO/TODO.md` files. These files list available work: bugs, features, cleanup tasks, documentation work, protocol work, and follow-up items.

Commitment Ledger observes those TODO files and records when people make promises against that work.

A TODO item says:

> This work exists.

A commitment says:

> I promise to do this work by this date.

Evidence says:

> This is what happened.

Assessment says:

> The promise was kept, partially kept, broken, superseded, refused, delegated, or expired unassessed.

This design follows the PromiseGrid mindset: promises, evidence, local assessment, durable records, and explicit commitment boundaries are first-class concepts. PromiseGrid app guidance emphasizes that apps should model promises, evidence, fulfillment, refusal, local trust, and protocol-specific meaning rather than flattening everything into ordinary request/response or task tracking.

---

2. Language and Implementation

Language: Go

Reasons:

- The team already writes Go.
- Go produces simple command-line binaries.
- Go fits the team's Unix-style tooling preference.
- Go can read Git repositories, JSON, Markdown, and local files easily.
- The first version should be CLI-first and safe.

---

3. Major Design Decisions

3.1 Observe Source Repositories Only

Commitment Ledger must not edit tracked repositories in v0.1.

It observes:

- repo name
- branch
- commit hash
- `TODO/TODO.md`
- TODO IDs
- TODO titles
- TODO detail files
- checked / unchecked state
- subtask state

It does not mark TODOs done. Humans or normal repo workflows do that.

3.2 Ledger Data Lives in the `commitment-ledger` Repo

Tracked repos remain unchanged.

All Commitment Ledger records live in the `commitment-ledger` repository.

3.3 Only the Promiser Can Create a Commitment

A person can only promise for themselves.

Allowed:

```
JJ promises to complete TODO-binap by 2026-07-01.
```

Not allowed:

```
Steve promises that JJ will complete TODO-binap.
```

Steve may request, suggest, review, or assess, but JJ must create JJ's own commitment.

3.4 Multiple People May Commit to the Same Work

The same TODO can have multiple commitments.

Example:

```
JJ promises to implement TODO-binap.
Steve promises to review TODO-binap.
Carol promises to test TODO-binap.
```

These are separate promises against related work.

3.5 Due Dates Are Required in v0.1

Every commitment must have a due date.

If the due date passes and the promise has not been assessed, the status becomes:

```
expired_unassessed
```

Important:

```
expired_unassessed does not automatically mean broken.
```

A later human or agent assessment may mark it:

- kept
- partially_kept
- broken
- superseded
- extended

This matches the PromiseGrid-style distinction between promise expiration, evidence, and later assessment. The guide treats expiry as part of the promise lifecycle and treats break/timeout evidence as assessable evidence rather than a universal automatic verdict.

---

4. Supported TODO Formats

4.1 Legacy Numeric Format

Example:

```
001 - Add route support
002 - Add malformed packet tests
003 - Fix accounting bug
```

Work ID:

```
001
002
003
```

4.2 Proquint TODO Format

Example:

```
[ ] TODO-binap - Lock and flesh out README outline (`TODO/TODO-binap-readme-
outline-lock.md`)
```

Work ID:

```
TODO-binap
```

Detail file:

```
TODO/TODO-binap-readme-outline-lock.md
```

4.3 Completion Format

Unchecked means open:

```
[ ] TODO-binap - Lock and flesh out README outline
```

Checked means complete:

```
[x] TODO-binap - Lock and flesh out README outline
```

Struck-through TODOs may also be treated as complete if the team uses that style in older repos.

---

5. Subtasks

Commitment Ledger should support subtasks in v0.1.

A top-level TODO may have a detail file:

```
TODO/TODO-binap-readme-outline-lock.md
```

That detail file may contain subfeatures, bugs, or subtasks.

The parser should discover checkboxes inside the detail file.

Example:

```
# TODO-binap README outline lock

## Tasks

- [ ] 1. Lock README headings
- [ ] 2. Add App Devs section
- [ ] 3. Add Kernel Devs section
- [ ] 4. Add citations
```

Subtask IDs may be inferred:

```
TODO-binap/1
TODO-binap/2
TODO-binap/3
TODO-binap/4
```

If headings use deeper numbering:

```
2.1
2.2
2.3
```

then subtask IDs become:

```
TODO-binap/2.1
TODO-binap/2.2
TODO-binap/2.3
```

5.1 Commitment Target Rules

A commitment may target:

```
repo/branch/TODO-binap
```

or specific subtasks:

```
repo/branch/TODO-binap/2.1
repo/branch/TODO-binap/2.2
```

If a person promises the parent TODO, all relevant required subtasks must be complete before the commitment can be assessed as kept.

If a person promises only one subtask, only that subtask must be complete.

---

6. Repository Tracking

Commitment Ledger should use a repository configuration file.

Suggested file:

```
config/repos.json
```

Example:

```
{
  "repos": [
    {
      "name": "gdoctools",
      "provider": "github",
      "url": "git@github.com:example/gdoctools.git",
      "local_path": "/home/jj/lab/gdoctools",
      "branch": "main",
      "todo_file": "TODO/TODO.md",
      "enabled": true
    },
    {
      "name": "wire-lab",
      "provider": "gitea",
      "url": "git@gitea.example.com:team/wire-lab.git",
      "local_path": "/home/jj/lab/wire-lab",
      "branch": "jj",
      "todo_file": "TODO/TODO.md",
      "enabled": true
    }
  ]
}
```

6.1 Git, GitHub, and Gitea

v0.1 should be Git-native, not GitHub-native or Gitea-native.

The system should work against local Git clones.

Provider fields such as `github` or `gitea` are metadata only in v0.1.

Later versions may use GitHub or Gitea APIs for PRs, reviews, and web links.

6.2 Branches Matter

The source identity must include branch.

This is important because:

```
repo/main/TODO-binap
```

and

```
repo/jj/TODO-binap
```

may represent different states.

JJ can keep a promise on her branch even if Steve later decides not to merge it into main.

---

7. Core Data Model

7.1 RepoSource

```
{
  "name": "gdoctools",
  "provider": "github",
  "url": "git@github.com:example/gdoctools.git",
  "local_path": "/home/jj/lab/gdoctools",
  "branch": "main",
  "todo_file": "TODO/TODO.md",
  "enabled": true
}
```

7.2 WorkItem

```
{
  "repo": "gdoctools",
  "branch": "main",
  "commit": "abc123",
  "work_id": "TODO-binap",
  "title": "Lock and flesh out README outline",
  "detail_file": "TODO/TODO-binap-readme-outline-lock.md",
  "status": "open",
  "source_file": "TODO/TODO.md",
  "first_seen": "2026-06-22T10:00:00-07:00",
  "last_seen": "2026-06-22T10:00:00-07:00"
}
```

7.3 WorkTarget

A WorkTarget is the exact thing promised.

Examples:

```
gdoctools/main/TODO-binap
gdoctools/main/TODO-binap/2.1
wire-lab/jj/003
wire-lab/jj/003/2.2
```

7.4 Commitment

```
{
  "commitment_id": "COMMITMENT-20260622-jj-001",
  "promiser": "JJ",
  "repo": "gdoctools",
  "branch": "main",
  "targets": [
    "gdoctools/main/TODO-binap/2.1",
    "gdoctools/main/TODO-binap/2.2"
  ],
  "promise_text": "I promise to complete TODO-binap subtasks 2.1 and 2.2.",
  "created_at": "2026-06-22T10:00:00-07:00",
  "due_date": "2026-06-28",
  "status": "open"
}
```

7.5 Evidence

```
{
  "evidence_id": "EVIDENCE-20260624-001",
  "commitment_id": "COMMITMENT-20260622-jj-001",
  "evidence_type": "todo_checked",
  "repo": "gdoctools",
  "branch": "main",
  "commit": "def456",
  "target": "gdoctools/main/TODO-binap/2.1",
  "observed_at": "2026-06-24T09:00:00-07:00",
  "notes": "Subtask 2.1 checked off in TODO detail file."
}
```

Evidence types:

```
todo_checked
todo_unchecked
commit_seen
manual_note
human_review
llm_review
assessment
```

A commit ID can be evidence.

A PR alone is not completion evidence in v0.1.

A checked TODO or checked subtask is completion evidence.

7.6 Assessment

```
{
  "assessment_id": "ASSESSMENT-20260628-001",
  "commitment_id": "COMMITMENT-20260622-jj-001",
  "assessor": "Steve",
  "status": "kept",
  "assessed_at": "2026-06-28T15:00:00-07:00",
  "basis": [
    "EVIDENCE-20260624-001",
    "EVIDENCE-20260625-002"
  ],
  "notes": "Both promised subtasks were checked off before the due date."
}
```

---

8. Commitment Statuses

Required statuses:

```
open
kept
partially_kept
broken
refused
delegated
superseded
```

```
extended
expired_unassessed
```

8.1 Status Meaning

`open`

The promise exists and the due date has not passed.

`kept`

All promised targets were completed and assessment agrees.

`partially_kept`

Some, but not all, promised targets were completed.

`broken`

The promise was assessed as not kept.

`refused`

The promiser explicitly refused or withdrew before completion.

`delegated`

The promiser delegated work, but this does not erase the original promise unless a new promise supersedes it.

`superseded`

A later commitment replaces this commitment.

`extended`

The commitment due date was extended by a later ledger event.

`expired_unassessed`

The due date passed, but no final assessment has been made.

---

9. Storage Architecture

Use both JSON and Markdown.

9.1 JSON for Machine-Readable Records

Suggested structure:

```
data/
  repos.json
  work_items.jsonl
  commitments.jsonl
  evidence.jsonl
  assessments.jsonl
  snapshots.jsonl
```

Use JSON Lines for append-friendly records.

9.2 Markdown for Human-Readable Records

Suggested structure:

```
records/
  commitments/
    COMMITMENT-20260622-jj-001.md
  assessments/
    ASSESSMENT-20260628-001.md
```

Example commitment Markdown:

```
# COMMITMENT-20260622-jj-001

Promiser: JJ
Repo: gdoctools
Branch: main
Due Date: 2026-06-28
Status: open

## Promise

I promise to complete TODO-binap subtasks 2.1 and 2.2.
```

```
## Targets

- gdoctools/main/TODO-binap/2.1
- gdoctools/main/TODO-binap/2.2

## Evidence

None yet.
```

9.3 Why Both?

JSON is reliable for programs.

Markdown is better for humans and LLMs.

The ledger should be auditable without requiring a database browser.

---

10. Proposed Architecture

10.1 CLI Layer

Package:

```
cmd/commitment-ledger
```

Commands:

```
commitment-ledger scan
commitment-ledger status
commitment-ledger commit
commitment-ledger evidence
commitment-ledger assess
commitment-ledger expire
commitment-ledger report
```

10.2 Internal Packages

Suggested Go layout:

```
commitment-ledger/
  cmd/
    commitment-ledger/
      main.go
  internal/
    config/
      config.go
    gitrepo/
      gitrepo.go
    todo/
      parser.go
      parser_test.go
    ledger/
      ledger.go
      jsonl.go
      markdown.go
    commitment/
      commitment.go
    evidence/
      evidence.go
    assessment/
      assessment.go
    report/
      report.go
  config/
    repos.json
  data/
    work_items.jsonl
    commitments.jsonl
    evidence.jsonl
    assessments.jsonl
    snapshots.jsonl
  records/
    commitments/
    assessments/
  README.md
```

10.3 Package Responsibilities

`config`

Reads repo tracker config.

`gitrepo`

Gets current branch and commit hash.

May checkout configured branch only if explicitly enabled.

Default behavior should not change branches.

`todo`

Parses `TODO/TODO.md` and TODO detail files.

Supports numeric and proquint formats.

Supports top-level items and subtasks.

`ledger`

Writes append-only JSONL records and Markdown records.

`commitment`

Defines commitment creation and validation.

`evidence`

Defines evidence records.

`assessment`

Defines assessment rules.

`report`

Generates human-readable summaries.

---

11. CLI Examples

11.1 Scan Repos

```
commitment-ledger scan --config config/repos.json
```

Output:

```
gdoctools main
Open work: 37
Completed work: 14
Subtasks discovered: 82
Commit: abc123
```

11.2 Create a Commitment

```
commitment-ledger commit
  --promiser JJ
  --repo gdoctools
  --branch main
  --target gdoctools/main/TODO-binap/2.1
  --target gdoctools/main/TODO-binap/2.2
  --due 2026-06-28
  --promise "I promise to complete TODO-binap subtasks 2.1 and 2.2."
```

11.3 Collect Evidence

```
commitment-ledger scan
```

If a target has been checked off, the system records evidence.

11.4 Expire Commitments

```
commitment-ledger expire
```

If due date has passed and commitment is still open:

```
status = expired_unassessed
```

11.5 Assess Commitment

```
commitment-ledger assess
  --commitment COMMITMENT-20260622-jj-001
  --assessor Steve
  --status kept
  --notes "All promised subtasks were checked off."
```

12. Reports

12.1 Repo Summary

```
Repo: gdoctools
Branch: main
Open TODOs: 37
Open subtasks: 82
Active commitments: 4
Expired unassessed: 1
Kept commitments: 12
```

12.2 Person Summary

```
Promiser: JJ
Open commitments: 3
Kept: 8
Partially kept: 1
Expired unassessed: 2
Broken: 0
```

12.3 Work Summary

```
Work: gdoctools/main/TODO-binap
Status: open
Subtasks: 4
Completed subtasks: 2
Commitments:
- JJ promised subtasks 2.1 and 2.2 by 2026-06-28
- Steve promised review by 2026-06-30
```

13. PromiseGrid Alignment

Commitment Ledger is not intended to be a full PromiseGrid kernel or protocol implementation in v0.1.

It is a grid app concept and local implementation prototype.

PromiseGrid concepts reflected in this app:

13.1 Promises

Each commitment is a promise made by a promiser.

13.2 Promisee

v0.1 may allow `promisee` to be optional or set to a team/project.

Future versions should model promisee explicitly.

13.3 Evidence

Checked TODOs, commit IDs, human review, and scan observations are evidence.

13.4 Local Assessment

The system should not assume one global truth.

A human or local reviewing agent assesses whether a commitment was kept.

13.5 Expiration

Due dates create expiration pressure.

Expiration does not automatically equal broken promise.

13.6 Durable Records

Ledger records are append-only and preserved over time.

13.7 Branch-Specific Reality

Work completed on one branch is not automatically completed on another branch.

This supports local views and avoids pretending every repo branch has one global state.

13.8 Future pCID Alignment

Future versions may define a PromiseGrid protocol spec for commitment records.

That future protocol may assign a pCID to the commitment format.

v0.1 should keep records structured enough to migrate into a later pCID-defined protocol.

14. v0.1 Scope

Must have:

- config-driven repo tracking
- Git branch and commit detection
- parse `TODO/TODO.md`
- parse numeric TODO format
- parse proquint TODO format
- parse detail file subtasks
- create commitment
- require due date
- scan for checked TODOs and subtasks
- record evidence
- expire open commitments into `expired_unassessed`
- manually assess commitments
- produce summary reports
- store JSONL and Markdown records

Should not have yet:

- web UI
- GitHub API integration
- Gitea API integration
- automatic PR review
- automatic merge detection
- automatic editing of TODO files
- global trust score
- full PromiseGrid protocol implementation

---

15. Future Scope

v0.2:

- web dashboard
- visual dependency graph
- commitment dependency edges
- GitHub/Gitea web links
- LLM-assisted review suggestions
- richer reports

v0.3:

- PromiseGrid message format
- pCID-defined commitment protocol
- signed commitment records
- capability tokens for delegated authority
- cross-repo dependency graphs

v1.0:

- distributed Commitment Ledger
- PromiseGrid-native commitment protocol
- local trust/accounting views
- multi-party commitment workflows
- durable evidence exchange between peers

---

16. Hopes and Dreams

Commitment Ledger starts as a practical tool for SteveGT-style repositories.

It helps answer:

- What work exists?
- Who promised to do what?
- By when?
- What evidence exists?
- What happened?
- What is expired but unassessed?
- What was actually kept?

The larger dream is that Commitment Ledger becomes an early PromiseGrid app that demonstrates why promises are more powerful than task assignments.

A task board says:

> This task exists.

Commitment Ledger says:

> This work exists, someone made a promise about it, evidence was collected, and the outcome is preserved.

That is the important shift.